NUST CHIP DESIGN CENTRE

RISC-V Assembly

EXERCISE # 03
RISCV Assembly Tasks

Name	*Muddassir Ali Siddiqui*
Instructor	*Sir Imran & Sir Umer*
Date	*19th Aug 2025*

1. In-Lab Tasks: (Write your lab task & screenshots here)

i. Task 1:

```
# Question # 01
.data
# # define constant n = 0x7FF (fits in 12-bit imm)
.equ n, 6

.text
j main

fact:                   # int fact(int n)
    addi sp, sp, -8       # make stack frame
    sw ra, 0(sp)          # save return address
    sw s0, 4(sp)          # save s0
    mv s0, a0             # move n into s0

    bge s0, t0, Recurse    # if (n >= 1) go to Recurse
    li a0, 1              # return 1 (base case)
    j Epilogue

Recurse:
    addi a0, s0, -1       # a0 = n-1
    jal ra, fact          # call fact(n-1)
    mv t1, a0             # store result of fact(n-1) in x6
    mul a0, s0, t1        # a0 = n * fact(n-1)

Epilogue:
    lw ra, 0(sp)          # restore return address
    lw s0, 4(sp)          # restore s0
    addi sp, sp, 8        # pop stack
    jr ra                 # return a0
```

```
main:
    li a0, n              # j = fact(5)
    addi t0, x0, 1
    jal ra, fact          # call fact
    mv s0, a0             # save result in s0
```

ii. Task 2:

```
# Question # 02
.data
# define constant n = 0x7FF (fits in 12-bit imm)
.equ n, 6

.text
jal main

fib:                      # int fib(int n)
    addi sp, sp, -12      # make stack frame (need extra space)
    sw ra, 0(sp)          # save return address
    sw s0, 4(sp)          # save s0
    sw s1, 8(sp)          # save s1
    mv s0, a0             # s0 = n

    beq s0, x0, Case0     # if (n == 0) return 0
    addi t0, x0, 1
    beq s0, t0, Case1     # if (n == 1) return 1

    # ---- Recursive case ----
    addi a0, s0, -1       # a0 = n-1
    jal ra, fib           # call fib(n-1)
    mv s1, a0             # save result of fib(n-1) in s1

    addi a0, s0, -2       # a0 = n-2
    jal ra, fib           # call fib(n-2)
    add a0, s1, a0        # a0 = fib(n-1) + fib(n-2)
    j Epilogue

Case0:
    li a0, 0              # return 0
    j Epilogue

Case1:
    li a0, 1              # return 1
```

```
Epilogue:
   lw ra, 0(sp)          # restore return address
   lw s0, 4(sp)          # restore s0
   lw s1, 8(sp)          # restore s1
   addi sp, sp, 12       # pop stack
   jr ra                 # return a0

main:
   li a0, n              # j = fib(6)
   jal ra, fib           # call fib
   mv s0, a0             # save result in s0
```